Aman Raj Verma

Software Engineer · Backend (Java, Spring Boot) · Applied AI (LLM Agents, RAG) · Gwalior, India
+91 8948810007 · amanrajverma003@gmail.com · github.com/ARV0007 · linkedin.com/in/arv14 · Portfolio: arv0007.github.io

SUMMARY

Backend engineer (M.Tech CSE, ABV-IIITM, 2027) who builds Java/Spring Boot systems and LLM agents, and measures them. Built a payment gateway with an agentic fraud-investigation layer (LangGraph, MCP) that caught 6/6 planted incidents, and a multi-agent AI app builder with RAG passing 20/20 of its evals. Mockito and Apache Kafka contributor; Stryker intern.

EDUCATION

Atal Bihari Vajpayee IIITM (ABV-IIITM), Gwalior, India
M.Tech., Computer Science and Engineering | 2025 – 2027 (Expected) | Coursework: Machine Learning, Deep Learning, DSA

Madan Mohan Malaviya University of Technology (MMMUT), Gorakhpur, India
B.Tech., Mechanical Engineering | 2020 – 2024 | *moved to CS by qualifying GATE CS*

TECHNICAL SKILLS

- **Languages:** Java, Python, SQL, C++, JavaScript · **Core CS:** Data Structures & Algorithms, OOP, DBMS, OS, CN
- **Backend:** Spring Boot, Spring Security, JPA/Hibernate, FastAPI, REST APIs, JWT, Apache Kafka, Microservices, Event-Driven Architecture, Distributed Systems, Scalability, Concurrency, Multithreading, Design Patterns, System Design
- **GenAI:** Agentic AI, LLM agents, multi-agent, RAG, vector search, hybrid search, reranking, tool calling, embeddings, prompt engineering, LLM evals, guardrails, LangChain, LangGraph, Hugging Face, Gemini API, OpenAI-compatible APIs
- **Databases & Storage:** PostgreSQL, pgvector, Qdrant, Redis, S3 (MinIO), Flyway
- **ML & Signals:** PyTorch, scikit-learn, 1D CNNs, HMMs, time-series classification, sensor signal processing, statistics
- **DevOps & Testing:** Docker, Kubernetes, CI/CD (GitHub Actions), Prometheus, Grafana, JUnit 5, pytest, Git, Render

WORK EXPERIENCE

Stryker | Software Engineering Intern | Remote/Gurugram, India | May 2026 – Present
Wearable Sensor Activity & Posture Recognition, in collaboration with ABV-IIITM Gwalior (Sep 2026 – Present)

- Planned how data for a wrist/ring wearable is collected, labelled and evaluated, after reviewing 24 public datasets.
- Built a Python human activity recognition pipeline (PyTorch, scikit-learn, 1D CNN, HMM) on 4 public datasets (about 200 people, 3,900 hours). All 7 experiments were planned up front, with leave-one-subject-out testing and bootstrap CIs.
- Gave the team concrete sensor specs. A gyroscope lifted activity recognition by **5.6 points** (macro F1) and transition detection by **3 points**, and 20 Hz sampling showed no measurable loss against 100 Hz.
- Found that lab-trained models lose about **half their accuracy** in real homes. Orientation-independent features won back a quarter of that gap, which made the case for a pilot study on the real device.

PROJECTS

Transakt + Sentinel | *Payments + Fraud AI* | Live Demo: arv0007.github.io/transakt, arv0007.github.io/sentinel
GitHub: github.com/ARV0007/transakt, github.com/ARV0007/transakt-sentinel
Java, Spring Boot, PostgreSQL, Redis, Kafka · Python, LangGraph, LangChain, MCP, FastAPI, scikit-learn, Hugging Face

- Stopped double charges from concurrent retries with idempotency keys, enforced by a PostgreSQL unique constraint.
- Kept money as integer paise in an append-only double-entry ledger. Hashed API keys or JWTs guard the API, Redis handles per-merchant rate limiting, and Redis caching made key lookups **3x** faster, falling back to PostgreSQL if Redis is down. A transactional outbox, Kafka and webhook retries deliver events at least once.
- Built Sentinel, three Python microservices that watch for fraud. An ML risk scorer checks every settled payment from Kafka in about 1 ms; a LangGraph agent investigates flagged ones via 6 read-only MCP tools, then waits for an analyst.
- Caught **6/6** planted fraud incidents with **0 false cases** on 20 normal merchants (precision 0.947) in a simulated run. Only 3% of traffic reached the LLM, and every number in its write-up is checked against the evidence.
- Made Redis-backed scoring **10x** faster (355 to 3,505 payments/s) by batching each Kafka poll. An atomic Redis Lua script kept all **200 of 200** concurrent payments across 2 replicas, where a naive read-then-write version kept 2.
- Added gated retraining from analyst decisions, an A2A endpoint and Prometheus/Grafana; **150 tests** run in CI.

ForgeFlow | *Multi-Agent App Builder* | Live Demo: forgeflow-7m08.onrender.com | GitHub: github.com/ARV0007/forgeflow
Java 21, Spring Boot 4, Gemini API, PostgreSQL, pgvector, Qdrant, Redis, Kafka, S3, Kubernetes, Docker, MCP, Python

- Built a Lovable-style app builder in Java, no AI framework: describe an app or paste a screenshot, get a running app. The agent can't finish until its code passes a build check (up to 3 repair rounds); a vision model then reviews the preview.
- Made it multi-agent: a planning builder, a reviewer agent that reads the diff and can send work back, and a read-only auditor. Deletes need human approval; model fallback and guardrails cover prompt injection and SSRF.
- Took code search to **87%** top-1 and **99%** top-3 on a 70-question benchmark I wrote, with hybrid RAG (pgvector/Qdrant + full-text, rank fusion, LLM rerank, agentic query rewriting) that also powers cited Q&A and chat memory.
- Hit **20/20** passing apps on a golden eval set in 3 of 3 runs, at about 11 s, 7.5k tokens and $0.003 each; 5/5 follow-ups landed as targeted edits. LLM-as-judge and trajectory checks sit beside **290 tests** on real Postgres, Redis, Qdrant and S3.
- Built to scale horizontally: gateway failover, stateless API replicas, agents on Java virtual threads, Kafka-driven indexing.

OPEN SOURCE AND WRITING

- **Mockito** (PR #3859, open): fixes issue #345, a bug open since 2016 where `ArgumentCaptor` captured millions of duplicates under `after()`. The project's lead maintainer endorsed the approach in review.
- **Apache Kafka** (PR #23926, open): fixes KAFKA-9998, where `KafkaProducer.close(timeout)` could block forever.
- **Technical blog** (arv0007.github.io/blog): write-ups on the Kafka fix, safe API-key caching and a Redis race.